
Lista Duplamente Encadeada

Prof. Fábio Medeiros Faria

Estrutura

```
5 struct produto {
6     int codigo;
7     int quantidade;
8     float valorUnitario;
9 };
10
11 typedef struct produto Produto;
12
13 struct elemento {
14     Produto produto;
15     struct elemento *anterior;
16     struct elemento *proximo;
17 };
18
19 typedef struct elemento Elemento;
20
21 typedef struct elemento* Lista;
```

Criar Lista

```
78  Lista* criarLista() {
79
80      Lista *lista = (Lista*) malloc(sizeof(Lista));
81
82      if(lista != NULL) {
83          *lista = NULL; //lista vazia
84      }
85
86      return lista;
87 }
```

Inserir Início

```
201 int inserirInicio(Lista *lista, Produto novoProduto) {
202
203     if(lista == NULL) {
204         printf("Nao existe lista");
205         return 0;
206     }
207
208     Elemento *elemento = (Elemento*) malloc(sizeof(Elemento));
209
210     if(elemento == NULL) {
211         printf("Nao foi possível alocar memória");
212         return 0;
213     }
214
215     elemento->produto = novoProduto;
216     elemento->proximo = (*lista);
217     elemento->anterior = NULL;
218
219     //lista não é vazia
220     if(*lista != NULL) {
221         (*lista)->anterior = elemento;
222     }
223
224     *lista = elemento;
225     return 1;
226 }
```

Iterar a Lista

```

258 void imprimirListaProdutos(Lista *lista) {
259
260     if (lista == NULL) {
261         printf("Imprimir: lista inexistente");
262         return;
263     }
264
265     printf("\n...: Lista de Produtos :...");
266     printf("\nCodigo | Qtd | ValorUni");
267     printf("\n-----");
268
269     Elemento *aux = *lista;
270
271     while(aux != NULL) {
272         printf("\n%d\t%d\t%.2f", aux->produto.codigo,
273             aux->produto.quantidade,
274             aux->produto.valorUnitario);
275
276         aux = aux->proximo;
277     }
278     printf("\n-----");
279 }

```

Uso Inicial

```
35  int main() {  
36  
37      Produto p1 = {1, 10, 11.99};  
38      Produto p2 = {2, 5, 19.87};  
39      Produto p3 = {3, 20, 99.77};  
40      Produto p4 = {4, 1, 100.01};  
41      Produto p5 = {5, 3, 93.01};  
42  
43  
44      Lista *listaProdutos = criarLista();  
45  
46      inserirInicio(listaProdutos, p1);  
47      inserirInicio(listaProdutos, p2);  
48      inserirInicio(listaProdutos, p3);  
49  
50      imprimirListaProdutos(listaProdutos);  
51  
52      return 0;  
53  }
```


Inserir Fim

```
145 int inserirFim(Lista *lista, Produto novoProduto) {
146
147     if(lista == NULL) {
148         printf("Inserir Fim: lista não existe!!");
149         return 0; //false
150     }
151
152     Elemento *elemento = (Elemento*) malloc(sizeof(Elemento));
153
154     if(elemento == NULL) {
155         printf("Inserir Fim: nao foi possivel alocar memoria :( ");
156         return 0; //false
157     }
158
159     elemento->produto = novoProduto;
160     elemento->proximo = NULL; //aponta para o finzinho
161
162     //lista eh vazia?
163     if( (*lista) == NULL) {
164         elemento->anterior = NULL;
165         *lista = elemento;
166     } else {
167         Elemento *aux = *lista;
168         //procura ultimo elemento
169         while(aux->proximo != NULL) {
170             aux = aux->proximo;
171         }
172
173         aux->proximo = elemento;
174         elemento->anterior = aux;
175     }
176     return 1;
```

Remover Início

```
226 int removerInicio(Lista *lista) {
227
228     if(lista == NULL) {
229         return 1;
230     }
231
232     if(*lista == NULL) {
233         return 1;
234     }
235
236     Elemento *aux = *lista;
237     *lista = aux->proximo;
238
239     if(aux->proximo != NULL) {
240         aux->proximo->anterior = NULL;
241     }
242
243     free(aux);
244     return 1; //true = sucesso removeu
245 }
```


Remover Fim

```
139 int removerFim(Lista *lista) {
140
141     if(lista == NULL) {
142         return 1;
143     }
144
145     if(*lista == NULL) {
146         return 1;
147     }
148
149     Elemento *aux = *lista;
150
151     //encontra o ultimo elemento
152     while(aux->proximo != NULL) {
153         aux = aux->proximo;
154     }
155
156     //ultimo eh tambem o primeiro?
157     if(aux->anterior == NULL) {
158         *lista = aux->proximo;
159     } else {
160         aux->anterior->proximo = NULL;
161     }
162
163     free(aux);
164
165     return 1;
166 }
```